

CONTENTS

Four questions

OAuth

The pipe: TLS

Everything else

PGP

comms-mail today

S/MIME

Decisions to make

PGP or S/MIME?

Words you'll meet

A PRIMER FOR THE COMMS-MAIL OWNER

How mail gets secured, and what comms-mail should do about it

PGP, S/MIME, OAuth, TLS and the domain checks behind them each protect a different thing. This page explains each one plainly, shows where comms-mail stands today, and ends with the decisions that are yours to make.

Four questions

Email was designed with no security at all. Everything that protects it now was added later, in layers, and each layer answers a different question. Most confusion comes from mixing them up, so keep them apart:

Can someone on the network read or change it?

[TLS](#)

Stops a snooping Wi-Fi, a hostile network, anyone sitting between you and your server.

Can the mail servers read it?

[PGP or S/MIME encryption](#)

Stops your provider, the recipient's provider, anyone who breaks into either mailbox, old backups.

Did it really come from them, unchanged?

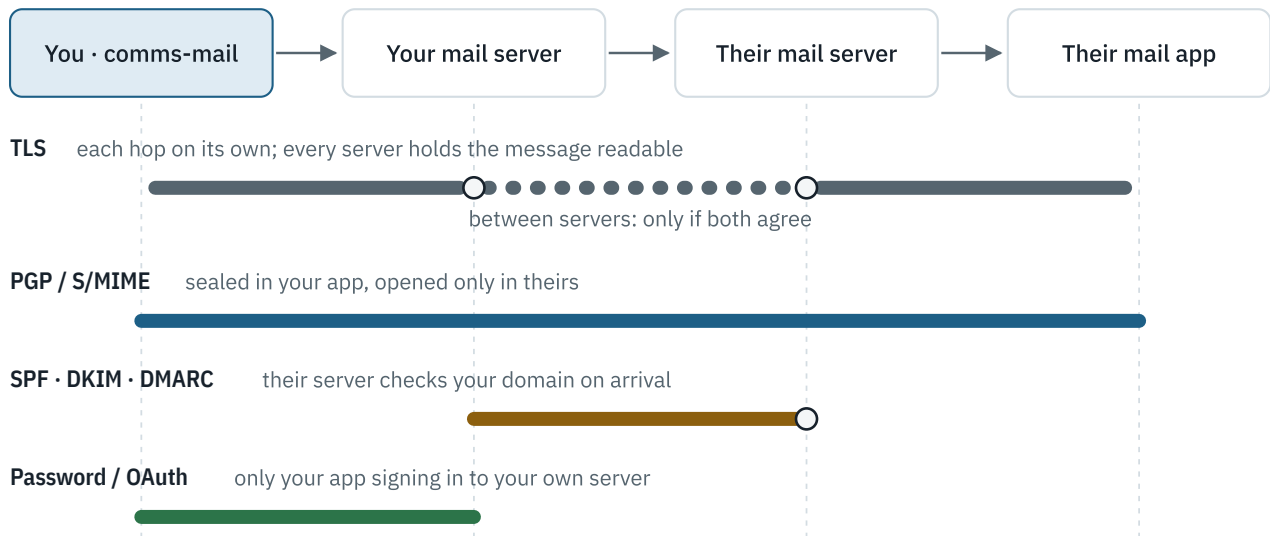
PGP or S/MIME signatures · SPF, DKIM, DMARC

Signatures prove the person; SPF, DKIM and DMARC prove only the domain. Together they fight forgery and phishing.

Who can sign in to your mailbox?

A password, or OAuth

OAuth keeps your password out of the app and lets you cut off one app without changing it.



One message, four protections. Only PGP and S/MIME cover the whole path; TLS is opened and re-sealed at every server along the way.

The pipe: TLS

TLS is the same encryption that puts the padlock on `https://` sites. For mail it protects each hop separately: your app to your server, and one server to the next. Every server along the way decrypts the message, stores it, and forwards it. TLS protects a message while it travels, not where it sits.

Two ways to start it

- **Implicit TLS:** encrypted from the very first byte. Ports `993` for IMAP and `465` for sending. The current standard (RFC 8314, 2018) recommends this one.
- **STARTTLS:** the app connects in plain text, then asks to switch to TLS (ports `143` and `587`). If an attacker deletes the server's offer to switch and the app carries on anyway, your password crosses the network readable. A careful app refuses to continue without TLS.

Checking the server is the server

The server proves its identity with a certificate. The app must check that the name matches, that a trusted authority signed it, and that it has not expired. An app that skips the check still encrypts, but to whoever answered, which may be an attacker.

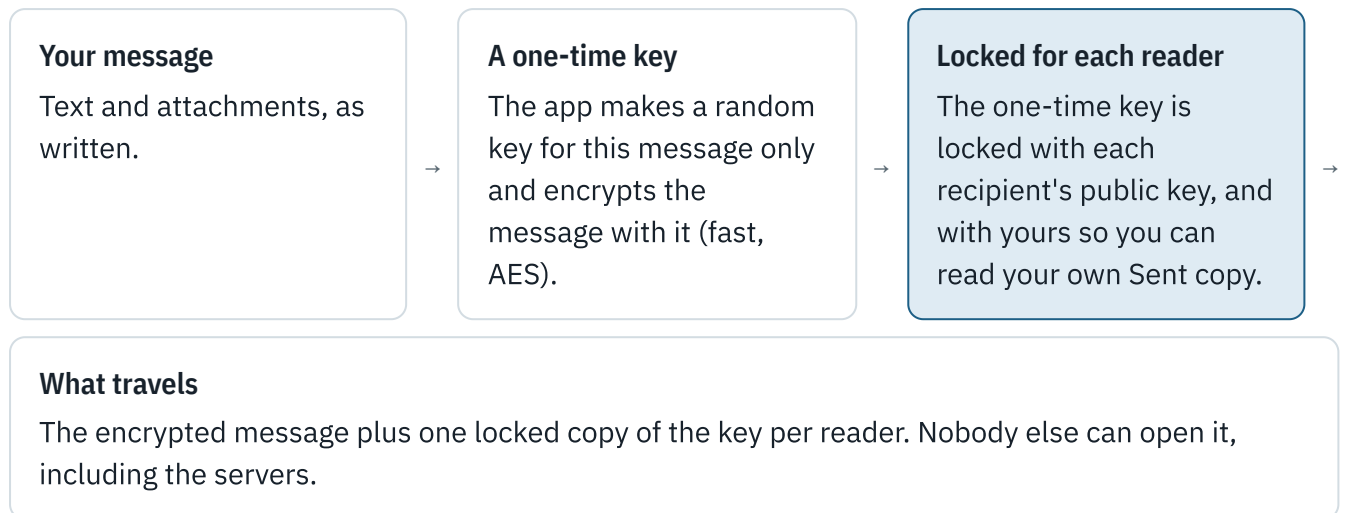
Between two mail servers, TLS is used only if both support it; otherwise they fall back to plain text. Domains can insist on TLS by publishing MTA-STS or DANE records. That is set up by whoever runs the domain, not by the mail app.

Your account. It reads mail from `mail.nchip.com:993` and sends through `mail.nchip.com:465`, both implicit TLS: the recommended setup.

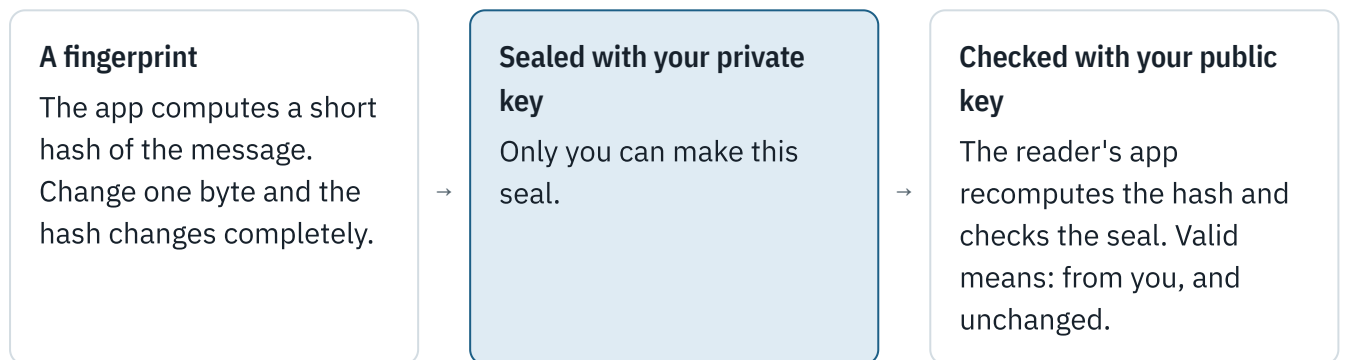
PGP

PGP gives every person two keys that belong together. The **public key** is handed out freely. The **private key** never leaves your computer. What the public key locks, only the private key opens; what the private key signs, anyone with the public key can check.

Encrypting



Signing



Signing hides nothing. It proves who wrote the message and that nobody altered it on the way.

The hard part: whose key is this?

Encryption itself is solved. Knowing that a public key really belongs to Bob, and not to someone pretending to be Bob, is the difficult part, and PGP has no central authority to vouch for keys. The ways people handle it:

- **Compare fingerprints** in person or over a channel you already trust. A fingerprint is a 40-character summary of the key.
- **WKD (Web Key Directory)**: the domain owner publishes keys on its own website at a standard address. Strong, because only the domain controls it.
- **keys.openpgp.org**: a key server that publishes a key only after its owner confirms the email address.
- **Autocrypt**: apps attach your public key to the headers of every mail you send, so the other side collects it automatically. Easy, but it trusts the first key it sees.
- **Web of trust**: people sign each other's keys. The original idea, now largely abandoned in practice.

What it looks like, and what it leaves visible

Modern PGP mail is **PGP/MIME** (RFC 3156): the message becomes a `multipart/encrypted` or `multipart/signed` part. An older style, inline PGP, puts `--- --BEGIN PGP MESSAGE-----` blocks in the text and copes badly with attachments and HTML.

PGP never hides who wrote to whom and when: From, To, Cc, the date and the servers stay readable. The subject is traditionally left readable too; newer apps move it inside the encryption ("protected headers") and show `...` outside.

Keys to keep safe

A private key is protected by a passphrase, or kept on a hardware key (YubiKey, Nitrokey, an OpenPGP smart card) that uses the key without ever giving it out. Lose the private key and old encrypted mail can never be read again. A **revocation certificate**, made once and stored somewhere safe, tells everyone a lost or stolen key must no longer be used.

The standard is OpenPGP: RFC 4880 from 2007, replaced in 2024 by RFC 9580 with modern algorithms. Well-known implementations: GnuPG (`gpg`), Sequoia, RNP (built into Thunderbird since version 78), and Proton's Go library, `go-crypto` .

S/MIME

S/MIME uses the same idea as PGP: a public key to lock and check, a private key to open and sign, the same one-time-key trick for encryption. On the wire a signed message carries an `application/pkcs7-signature` part (often shown as `smime.p7s`) and an encrypted one is an `application/pkcs7-mime` part. The difference is who vouches for a key.

Certificates and the authorities that issue them

Your public key comes inside a **certificate**: a small signed document saying "this key belongs to ada@example.com", issued by a **certificate authority** (CA), the same kind of company that issues certificates for websites. Operating systems and mail apps ship with a list of CAs they trust, so a signed message shows as verified without anyone comparing fingerprints.

- **Where you get one:** from a public CA (some give free email-only certificates, most charge), or from your employer's own CA. Companies, governments and the military often issue them on smart cards.
- **What you receive:** a `.p12` / `.pfx` file holding your certificate and private key, protected by a password.
- **How long it lasts:** a certificate always expires. Public CAs follow the CA/Browser Forum's S/MIME rules (since 2023), which cap it at about two years. Keep old private keys: they still open old mail.
- **How you get someone else's:** their certificate rides along in every message they sign. The usual first step is that they send you a signed mail; your app keeps their certificate, and from then on you can encrypt to them. There is no global directory, though companies may publish certificates internally.

S/MIME is built into Outlook, Apple Mail and iPhone Mail, Thunderbird, and KMail (through GnuPG's `gpgsm`). Gmail supports it for some Google Workspace editions.

PGP or S/MIME?

They do the same job and do not work together: a PGP user and an S/MIME user cannot exchange encrypted mail. Supporting both lets comms-mail talk to both worlds. Which to build first depends on who you write to.

	PGP	S/MIME
Who vouches for a key	You: fingerprints, WKD, key servers, Autocrypt	A certificate authority
Cost	Free	Free to paid; issued by the employer at work
Getting started	Make a key pair in the app	Get a certificate from a CA, import the .p12
Common among	Open-source and security people, journalists, Proton Mail users	Companies, government, Outlook and Apple users
Other people's keys	Key servers, WKD, Autocrypt, or they send it	From a signed message they send you
Expiry	Optional	Always, about 1–2 years
Hides who talks to whom	No (subject: optionally)	No
Hardware keys	OpenPGP cards, YubiKey	Smart cards (PIV), YubiKey

EFAIL, the lesson for both. In 2018 researchers showed that a mail app which decrypts a message and then renders it as HTML while loading remote content can leak the decrypted text to an attacker. The defences: never load remote content in decrypted mail, never stitch a decrypted part together with unencrypted parts into one HTML view, and treat any decryption or integrity error as a hard failure. comms-mail already blocks remote content by default, which is the most important of the three.

OAuth

With a password, every mail app you use holds the key to your whole account. If one leaks it, the thief has your mail, and often everything else that password opens. OAuth lets an app use your mailbox without ever seeing your password. Google and Microsoft have been switching password sign-in off for mail apps for exactly this reason.

What happens when you sign in

- 1 **COMMS-MAIL**
You press Sign in; comms-mail opens your browser at Google's or Microsoft's own sign-in page.
- 2 **YOU, IN THE BROWSER**
You sign in there with your password and second factor. comms-mail never sees either.
- 3 **GOOGLE OR MICROSOFT**
It asks whether comms-mail may read and send your mail. You approve.
- 4 **BROWSER → COMMS-MAIL**
The browser hands comms-mail a one-time code on a local address such as `http://127.0.0.1:port`.
- 5 **COMMS-MAIL → GOOGLE OR MICROSOFT**
comms-mail trades the code for two tokens: an **access token** that works for about an hour, and a **refresh token** that lasts and fetches new access tokens.
- 6 **COMMS-MAIL → MAIL SERVER**
It signs in to IMAP and SMTP with the access token instead of a password (the `XOAUTH2` or `OAUTHBEARER` sign-in methods).

PKCE makes the code from step 4 useless to anyone else: in step 5 comms-mail proves it is the same app that started the sign-in. That matters on a desktop, where another program could see the code go by.

After sign-in, the **refresh token** is the valuable secret. It must be stored encrypted, and you can revoke it at any time from your Google or Microsoft account page without changing your password.

The client ID: the pinned decision

Google and Microsoft only talk to apps they know. Every app is registered with them and gets a **client ID**. A desktop app cannot keep a secret, so the client ID is public; PKCE is

what protects the sign-in. The question is whose registration comms-mail uses.

ROUTE	HOW IT WORKS	CATCH
Your own registration	You register an app in Google Cloud Console or Microsoft Entra and paste its client ID into comms-mail, which already supports this. Free, no review.	A Google app left in "Testing" issues refresh tokens that expire after 7 days, so you sign in again weekly. Publishing it means Google's verification for full mail access. A Google Workspace organisation can mark it Internal and skip review. Some Microsoft 365 organisations require an admin to approve new apps.
One built into comms-mail	comms-mail ships its own registration, as Thunderbird does. Best for anyone else who uses comms-mail.	Google treats full Gmail access as a restricted permission: the app must pass verification and a yearly independent security assessment (CASA), which costs time and money. Microsoft asks for publisher verification, which is free.
App passwords	A personal Google account with 2-Step Verification can make a password just for one app.	It is still a password with full access. Workspace admins can turn it off, and Microsoft 365 no longer offers it for mail.

Which accounts need it. Gmail and Google Workspace, Microsoft 365 and Outlook.com. Microsoft turned off password sign-in for IMAP and POP in Exchange Online in 2022 and announced the same for sending (SMTP) during 2026. Your current account on mail.nchip.com signs in with a password and offers no OAuth, so none of this applies to it today.

Everything else

Is the sender who they claim? SPF, DKIM, DMARC

Three records a domain publishes so receiving servers can check mail that claims to come from it:

- **SPF:** which servers may send mail for the domain.

- **DKIM**: the sending server signs each message with the domain's key.
- **DMARC**: what to do when those fail, and whether they match the From address you actually see.

Your server runs these checks on arrival and writes the outcome into an

`Authentication-Results` header. A mail app can read that header and warn you: "claims to be from paypal.com but failed the check". These prove the domain, not the person, and cannot catch a look-alike domain (`paypa1.com`) or a display name like "PayPal" in front of a stranger's address. An app can flag those separately.

Tracking and remote content

An image loaded from the sender's server tells them when you opened the message, from which IP address, and roughly with what. Newsletters use invisible one-pixel images for exactly this. Blocking remote content until you ask is also the main EFAIL defence.

Links and attachments

The text of a link and where it goes can differ, so a good app shows the real destination before opening it. Opening an attachment hands it to another program; some file types (desktop launchers, scripts, installers) run code when opened, and should be refused.

On your disk

A mail app keeps copies of your mail, and some secret (a password or a token) to sign in with. File permissions keep other users of the machine out. They do not stop a program that runs as you, which can read everything you can. Full-disk encryption protects a lost or stolen laptop; nothing in a mail app protects against malware already running under your account. The desktop keyring (GNOME Keyring, KWallet) keeps secrets encrypted until you log in, which protects them in backups and copied files.

Where comms-mail stands today

Read from the code on 28 September 2026. comms-mail is strict where the network is concerned and careful when showing mail. Its weak spot is the secrets it keeps on disk, and it has nothing yet for end-to-end encryption or for telling you whether a sender is genuine.

STRONG

The pipe (TLS), reading HTML, remote content, the local daemon.

WEAK

Secrets at rest: passwords in plain text, the token key beside the tokens.

MISSING

PGP, S/MIME, signature checks, sender warnings.

AREA	WHAT COMMS-MAIL DOES	STATE
THE PIPE		
Certificate checks	Always on, TLS 1.2 or newer, with no way to switch them off.	IN PLACE
STARTTLS stripping	If a server does not offer TLS the connection is refused, never continued in plain text. A password is never sent unencrypted (except to this machine itself).	IN PLACE
New accounts	Chooses implicit TLS from the port (993, 995, 465); STARTTLS for 143, 110, 587.	IN PLACE
SIGNING IN		
Passwords	Stored in plain text in <code>~/.config/comms-mail/mail.json</code> (readable only by you, mode 0600). The Add Account screen says so. No keyring.	WEAK
OAuth sign-in	Google and Microsoft, with PKCE and a local redirect, device-code as a fallback. IMAP and SMTP use <code>XOAUTH2</code> . POP has no OAuth.	IN PLACE
OAuth tokens	Encrypted (AES-256-GCM), but the key is written to <code>master.key</code> right beside them as well as to the keyring, so they are only as safe as a plain file.	PARTIAL
Client ID	You supply it (wizard or environment). None is built in.	YOUR CALL

AREA	WHAT COMMS-MAIL DOES	STATE
END TO END		
PGP	Not supported. An encrypted PGP message shows an empty body with <code>encrypted.asc</code> as an attachment.	MISSING
S/MIME	Not supported. An S/MIME-encrypted message (<code>smime.p7m</code>) shows its binary contents as the message text.	MISSING
Signed mail	Shows normally, with <code>signature.asc</code> or <code>smime.p7s</code> as an attachment. Never checked.	MISSING
WHO SENT IT		
SPF, DKIM, DMARC	The server's results are never read or shown.	MISSING
Look-alike senders	No warning for a display name that does not match its address, or for a Reply-To that differs from From. The list shows only the display name.	MISSING
READING		
Scripts	Never run: the HTML view has no script engine, and scripts, styles and frames are stripped first.	IN PLACE
Remote images	Blocked until you ask; Always for a sender. The sender allow-list trusts the From address, which can be forged.	IN PLACE
Image fetching	Only public addresses (the check is made on the address actually connected to), size and count limits, no cookies.	IN PLACE
Links	A confirmation shows the real destination first.	IN PLACE
Attachments	Launchers, scripts, installers and HTML are refused by file name; everything else goes to the desktop's opener. Opened copies in <code>~/ .data/comms-mail/open</code> and printed pages in <code>/tmp</code> are never cleaned up.	PARTIAL

AREA	WHAT COMMS-MAIL DOES	STATE
THIS MACHINE		
Mail on disk	<code>mail.db</code> , message files and the log are readable only by you, and not encrypted; that relies on full-disk encryption.	AS DESIGNED
The log	Records errors and which request failed, never its contents; passwords are not echoed.	IN PLACE
The daemon's socket	Only your user (or root) may connect. Any program running as you has the same access as the app: it can read mail and send as you. It can also point an account at a new server and the saved password goes along.	PARTIAL
Message-ID	Sent mail carries <code><number.your-address@uitoolkit></code> , which names the software and repeats your address.	PARTIAL
Header injection	Line breaks in addresses, subjects and file names are refused.	IN PLACE

Fixes that need no decision

- Show an S/MIME- or PGP-encrypted message as "encrypted, cannot be opened yet" instead of binary text or an empty body.
- Clean up opened attachments and printed pages once they are closed or old.
- Make sent Message-IDs look like everyone else's (`<random@your-domain>`), without the software name or your address.
- Ask for the password again when an account is pointed at a different server, instead of carrying the saved one over.
- Read the `Authentication-Results` header and warn on a failed DMARC or DKIM check, a display name that pretends to be another address, and a Reply-To on another domain. Trust the per-sender "Always show images" only when the sender passed those checks.

Decisions to make

Each of these changes what gets built. They are yours to make; each has a recommendation.

1. Where passwords and tokens live

Desktop keyring (GNOME Keyring or KWallet through the Secret Service): unlocked when you log in, encrypted on disk and in backups, and the way Evolution, Geary and KMail do it. A plain file stays only as the fallback on a desktop without a keyring.

A passphrase of your own: comms-mail encrypts its secrets with a passphrase you type when it starts, like Thunderbird's primary password. Works everywhere, but asks you every login.

Recommended: the keyring, with the passphrase as an option for machines without one. Either way, `master.key` stops being written next to the tokens.

2. PGP or S/MIME first

This depends on the people you write to. Colleagues on Outlook or Apple Mail in a company that issues certificates point to S/MIME. Open-source, security or Proton Mail contacts point to PGP.

Recommended: first, for both, recognise and check what arrives (signed mail shows as verified or not; encrypted mail says what it is). That is the common case and it shares the work. Then sending and decrypting for whichever your contacts use. Tell me who you exchange mail with and I will order it.

3. Whose crypto engine

GnuPG (`gpg` for PGP, `gpgsm` for S/MIME), the way KMail does it. Your keys stay in GnuPG, where `git` and `pass` use them too. Hardware keys, passphrase prompts and certificate handling come with it. It needs GnuPG installed, and comms-mail talks to it as a separate program.

Built in (pure Go, the way Thunderbird has its own engine): Proton's `go-crypto` for PGP and a third-party package for S/MIME, since Go has none of its own. Nothing to install, but comms-mail must then store private keys safely itself and needs its own key manager, and hardware keys are hard to support.

Recommended: GnuPG. Storing secrets is comms-mail's weakest spot today, and with GnuPG the most valuable secret, your private key, never becomes comms-mail's to guard. One engine covers both PGP and S/MIME. A built-in engine can come later for machines without GnuPG.

4. Searching encrypted mail

To search inside encrypted mail, comms-mail has to keep its decrypted text in the search index, which is a readable copy on disk of what was meant to be sealed. Without it, encrypted mail is found by sender, subject and date only.

Recommended: do not keep decrypted text by default; offer it as a setting for those whose disk is encrypted.

5. The OAuth client ID

Your own registration works today and costs nothing. A comms-mail registration matters only once other people use comms-mail with Gmail or Microsoft 365, and for Gmail it means Google's verification and a yearly paid security assessment.

Recommended: stay with your own registration while comms-mail is yours alone. Decide on a built-in one when others will use it. Is that the plan?

6. Sender warnings

Showing the server's SPF, DKIM and DMARC results, and flagging look-alike senders, costs little and catches most phishing that reaches an inbox. It needs no keys and works on every account, including yours.

Recommended: yes, and early. It is the most protection for the least work.

Words you'll meet

Public key	The half of a key pair you give out. Others use it to encrypt to you and to check your signatures.
Private key	The half that never leaves you. It opens what was encrypted to you and makes your signatures.
Fingerprint	A short, unique summary of a public key, used to confirm it is the right one.
Certificate	A public key plus a statement of whose it is, signed by a certificate authority (S/MIME, and websites).
Certificate authority	An organisation apps trust to vouch for certificates.
Session key	The random one-time key that actually encrypts a message; it is then locked for each reader.
Hash	A fixed-size digest of data. Any change to the data changes the hash.
Revocation	Declaring a key or certificate no longer valid, for example after it is lost.
Implicit TLS	A connection that is encrypted from the first byte (ports 993, 465).
STARTTLS	A plain connection that switches to TLS on request (ports 143, 587).
Client ID	The public name an app is registered under with Google or Microsoft.
Access token	A short-lived pass (about an hour) an app shows instead of a password.
Refresh token	A long-lived secret the app uses to get new access tokens. Revocable from your account page.
Scope	What an app asked permission to do, such as "read and send mail".
PKCE	A check that ties the sign-in code to the app that started the sign-in.
XOAUTH2	The IMAP and SMTP sign-in method that takes an access token.
Keyring	The desktop's store for secrets (GNOME Keyring, KWallet), unlocked when you log in.